

MicroInfer - Build Guide

Building a Transformer Inference Engine From Scratch — KV-Cache, Continuous Batching, and Quantization, Benchmarked Against vLLM

Project Specification & Build Guide

July 2026

- 1 Executive Summary
- 2 Chapter 1: What Problem Are We Actually Solving?
 - 2.1 1.1 The Problem, Precisely Stated
 - 2.2 1.2 The Three Techniques This Project Implements
 - 2.3 1.3 Why Build This Yourself Instead of Just Using vLLM
 - 2.4 1.4 What “Success” Looks Like for This Project
- 3 Chapter 2: Background Concepts You Need Before Starting
 - 3.1 2.1 The Attention Mechanism, Precisely
 - 3.2 2.2 Why Naive Generation Is Slow Without Caching
 - 3.3 2.3 Memory Layout: Why the KV-Cache Is the Actual Hard Part
 - 3.4 2.4 Batching With Variable-Length Sequences
 - 3.5 2.5 Quantization Basics
 - 3.6 2.6 Prerequisite Skills Checklist
- 4 Chapter 3: System Architecture
 - 4.1 3.1 High-Level Component Diagram
 - 4.2 3.2 Why This Ordering (Naive → Cached → Batched → Quantized)
 - 4.3 3.3 Technology Choices and Why
 - 4.4 3.4 Scope Boundary: What This Project Explicitly Does NOT Implement
- 5 Chapter 4: Phase-by-Phase Build Plan
 - 5.1 4.1 Phase 0 — Environment and Baseline (2-3 days)
 - 5.2 4.2 Phase 1 — Naive Forward-Pass Generation, No Cache (3-4 days)
 - 5.3 4.3 Phase 2 — KV-Cache Implementation (1-1.5 weeks)
 - 5.4 4.4 Phase 3 — Continuous Batching Scheduler (1.5-2 weeks)
 - 5.5 4.5 Phase 4 — Weight Quantization (1-1.5 weeks)
 - 5.6 4.6 Phase 5 — Rigorous Benchmarking Against vLLM (1-1.5 weeks)
- 6 Chapter 5: Benchmarking Methodology
 - 6.1 5.1 Why Methodology Matters More Than Results Here
 - 6.2 5.2 Fixed Variables (Must Be Identical Across All Comparisons)
 - 6.3 5.3 Metrics to Report (All of These, Not a Subset)
 - 6.4 5.4 Test Scenarios (Run All of These, Report All of Them)
 - 6.5 5.5 Statistical Rigor
 - 6.6 5.6 The Actual Analysis Write-Up Structure
- 7 Chapter 6: Expected Results and Honest Framing
 - 7.1 6.1 What You Should Expect to Find (Calibrating Expectations Before You Start)
 - 7.2 6.2 If Your Results Don’t Match These Expectations
 - 7.3 6.3 Why an Honest “This Didn’t Work” Result Is Still a Strong Deliverable
- 8 Chapter 7: Timeline Summary
- 9 Chapter 8: Repository Structure
- 10 Chapter 9: Interview Preparation — Turning This Project Into Talking Points

10.1 9.1 Questions You Should Be Able to Answer Cold

10.2 9.2 The Failure-Log Habit (Start This From Day One)

11 Appendix A: Glossary

12 Appendix B: Honest Caveats on This Document

1 Executive Summary

Project title: MicroInfer — a from-scratch transformer inference engine that implements the three techniques that make production LLM serving fast (KV-caching, continuous batching, and weight quantization), built without relying on an existing serving framework, and rigorously benchmarked against a real production system (vLLM) to produce an honest, defensible account of the performance gap and why it exists.

Why this project, specifically, for cracking elite AI research and FAANG-tier roles: every major AI lab and every large tech company runs LLM inference at scale, and the engineers who understand *why* inference is fast (not just how to call an API that makes it fast) are a small, valuable population. This project forces you to actually understand attention mechanics, memory layout, GPU utilization, and scheduling — the exact things an interviewer at Anthropic, OpenAI, Google DeepMind, or a FAANG infra team will probe. A polished demo that “just works” is common. A working implementation with a rigorous, honest comparison against the real thing, including where and why yours is slower, is rare and memorable.

What you will end up with: - A working Python/PyTorch inference engine, running a real small open-weight model (in the 1B–3B parameter range), that supports batched, cached, quantized generation. - A benchmark suite comparing your engine against vLLM (and optionally llama.cpp) on throughput (tokens/sec), latency (time-to-first-token, time-per-output-token), and memory footprint, across a range of batch sizes and sequence lengths. - A written analysis explaining every gap between your numbers and vLLM’s, backed by profiling data, not guesses. - A public GitHub repository with clean code, a proper README, reproducible benchmark scripts, and your written analysis — this is the actual artifact you show recruiters and bring up in interviews.

Time estimate: 6–9 weeks of focused part-time work (evenings/weekends around your coursework), broken into 6 phases in this document. It can be compressed if you can dedicate more hours per week — the phase breakdown in Chapter 5 gives you the dependency order so you know what can and can’t be parallelized.

2 Chapter 1: What Problem Are We Actually Solving?

2.1 1.1 The Problem, Precisely Stated

When you send a prompt to ChatGPT, Claude, or Gemini, the model doesn't process your entire conversation from scratch for every single word it generates. If it did, generating a 500-token response to a 2000-token conversation would require re-processing the entire 2000+ tokens of context 500 separate times — once per output token. That would make every conversation take minutes instead of seconds, and would make serving many users simultaneously prohibitively expensive.

Production inference systems solve this with a specific set of engineering techniques, and this project implements the three most important ones yourself, from first principles, so you understand exactly how and why they work — rather than only knowing them as configuration flags in someone else's library.

2.2 1.2 The Three Techniques This Project Implements

KV-Caching. In transformer attention, each new token needs to attend to all previous tokens' Key and Value projections. Without caching, you recompute these projections for the entire sequence on every single generation step. With caching, you compute each token's Key/Value once, store them, and reuse them for every subsequent step — turning an $O(n^2)$ re-computation problem per generation into an $O(n)$ incremental one.

Continuous Batching. Real production traffic has many users sending requests at different times, with different prompt lengths and different desired output lengths. Naive batching (waiting to collect a fixed batch, running it to completion, then starting the next batch) wastes enormous GPU capacity, because short requests in the batch finish early and then sit idle while longer ones finish. Continuous batching (sometimes called "in-flight batching") allows new requests to join a running batch and completed requests to leave it, on every single generation step, keeping the GPU busy nearly all the time instead of idling.

Quantization. Model weights are normally stored as 16-bit or 32-bit floating point numbers. Quantization reduces this precision (commonly to 8-bit or 4-bit integers) to shrink memory footprint and increase throughput, at the cost of some accuracy loss that must be measured and characterized, not assumed to be negligible.

2.3 1.3 Why Build This Yourself Instead of Just Using vLLM

This is the question every interviewer will ask, and the honest answer is the point of the project: **nobody should use a homemade inference engine in production when vLLM exists and is excellent.** The value of building this yourself is entirely educational and diagnostic — it's the difference between "I know that continuous batching improves throughput" and "I implemented a scheduler, watched it fail to improve throughput on my first attempt, found out why in the profiler, and fixed it." The second sentence is what survives a rigorous technical interview; the first one is what every other candidate says.

2.4 1.4 What "Success" Looks Like for This Project

Not: matching or beating vLLM's numbers. You will not beat a production system built by a dedicated team over years, and claiming otherwise would be either wrong or evidence you benchmarked incorrectly.

Actually: producing a working engine, a rigorous and reproducible benchmark, and — most importantly — a precise, profiler-backed explanation of every place your numbers differ from vLLM's, and why. An interviewer who asks "why is your engine only doing 30% of vLLM's throughput" should get a real answer ("vLLM uses PagedAttention to avoid memory fragmentation from variable-length sequences in the KV cache; I'm using naive padding, which wastes X% of memory and reduces the effective batch size I can fit") rather than a shrug.

3 Chapter 2: Background Concepts You Need Before Starting

You do not need to be a transformer expert before starting, but you do need these concepts solid before Phase 1, or you'll be debugging confusion rather than code.

3.1 2.1 The Attention Mechanism, Precisely

For a sequence of tokens, each token is projected into a Query (Q), Key (K), and Value (V) vector. Attention for a given token computes how much it should “attend to” every previous token, using:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

The critical detail for this project: when generating token $N+1$, its Query only needs to be compared against the Keys and Values of tokens $1...N$ — and those Keys and Values, for tokens $1...N$, are identical to what they were when you generated token N . This is exactly why caching them is valid and exactly why it saves so much recomputation.

3.2 2.2 Why Naive Generation Is Slow Without Caching

Without a KV-cache, generating token $N+1$ means re-running the entire forward pass over tokens $1...N+1$ from scratch, recomputing Keys and Values for all of them, even though tokens $1...N$'s Keys/Values haven't changed. Generating a 500-token response this way does roughly 500 forward passes over increasingly long sequences — the compute cost grows roughly quadratically with output length instead of linearly.

3.3 2.3 Memory Layout: Why the KV-Cache Is the Actual Hard Part

The KV-cache for one sequence, at one point in generation, is a tensor of shape `(num_layers, 2, num_heads, sequence_length, head_dim)` — the “2” is for K and V separately. For a 3B-parameter model with, say, 32 layers, 32 heads, head dimension 128, at a sequence length of 2048 tokens, in 16-bit precision, this cache alone occupies **several gigabytes per sequence**. This is why memory management for the KV-cache — not raw compute — is usually the actual bottleneck in production serving, and why vLLM's headline contribution (PagedAttention) is specifically a memory management technique, not a compute optimization. Understanding this is the single most important insight for this whole project, and for sounding credible in an interview about inference systems.

3.4 2.4 Batching With Variable-Length Sequences

Real requests have different prompt lengths and, once generation starts, different numbers of tokens already generated. Naive batching requires padding every sequence in the batch to the length of the longest one, which wastes both compute (attending to padding tokens, which must then be masked out) and memory (allocating cache space for padding that carries no information). Continuous batching solves the *scheduling* half of this problem (letting sequences join/leave the batch dynamically); PagedAttention-style memory management solves the *storage* half (allocating KV-cache memory in fixed-size blocks, like an OS's virtual memory pages, so sequences don't need contiguous, pre-allocated, padded memory). This project implements a simplified version of the scheduling half, and explicitly does not attempt to reimplement PagedAttention itself (see Chapter 4.4 for what a reasonable scope boundary looks like here).

3.5 2.5 Quantization Basics

Reducing a weight from 16-bit float to 8-bit integer requires a **scale factor** per group of weights, so that `int8_value * scale ≈ original_float_value`. The two practical choices for this project: - **Post-training quantization (PTQ)**: quantize an already-trained model's weights, no retraining needed. Simpler, some accuracy loss. - **Weight-only quantization**: keep activations in higher precision, only quantize the (much

larger) weight matrices. This is the standard approach for inference-time quantization and is what this project implements.

You do not need to implement quantization-aware training — that’s a training-time technique and out of scope here.

3.6 2.6 Prerequisite Skills Checklist

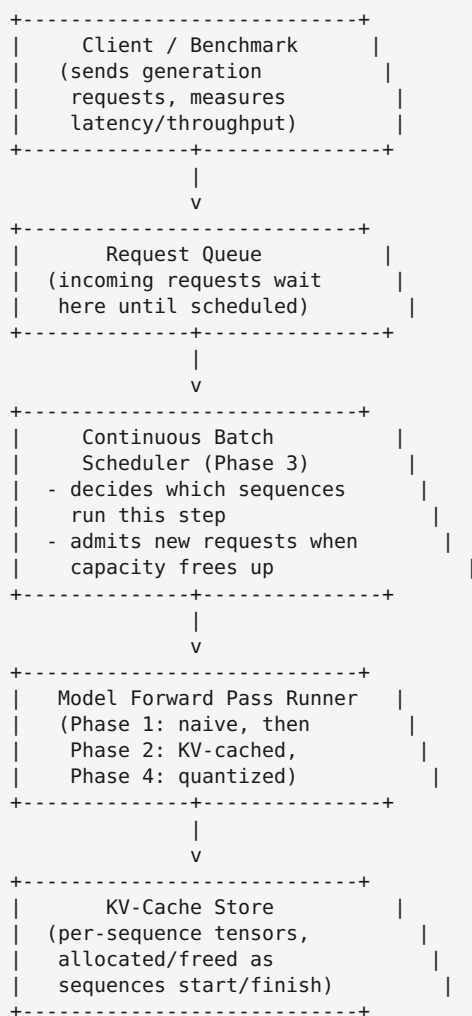
Before starting Phase 1, confirm you’re comfortable with:

- ☐ PyTorch tensor operations, especially `view / reshape`, indexing, and broadcasting.
- ☐ Reading and modifying a HuggingFace `transformers` model’s forward pass (you will need to hook into or reimplement parts of it).
- ☐ Basic CUDA concepts: what a kernel launch is, what GPU memory bandwidth vs. compute-bound means (you don’t need to write CUDA code, just understand these terms since profiling will surface them).
- ☐ Python’s `asyncio` or threading basics, for the batching scheduler in Phase 3.

If any of these are shaky, spend 2-3 days on them before Phase 1 — it will save far more time than it costs.

4 Chapter 3: System Architecture

4.1 3.1 High-Level Component Diagram



4.2 3.2 Why This Ordering (Naive → Cached → Batched → Quantized)

Each phase builds strictly on the previous one, and each phase is independently benchmarkable — meaning you get a working, measurable system at every stage, not just at the very end. This matters both for your own debugging (you can isolate which change caused a regression) and for your final write-up (you can show an incremental performance graph: naive → +caching → +batching → +quantization, each with its own measured improvement, which is far more compelling than a single before/after number).

4.3 3.3 Technology Choices and Why

- **Base framework: PyTorch**, not a from-scratch tensor library. Reimplementing autodiff or tensor operations is a different (and much larger) project; the value here is in the serving-layer engineering, not reimplementing PyTorch.
- **Base model: a small open-weight model in the 1-3B parameter range** (e.g., a small Llama-family or Qwen-family model) so that everything fits comfortably on a single consumer or cloud GPU and iteration is fast. Confirm current model availability and licensing terms before choosing, since specific model releases and their license terms change over time.
- **Quantization library: implement weight-only quantization yourself** for the core project (this is where the learning value is), but it's acceptable to use `bitsandbytes` as a reference/validation baseline to confirm your own implementation's accuracy loss is in a reasonable range.

- **Benchmark comparison targets: vLLM**, and optionally `llama.cpp` if you want a second, differently-architected comparison point (`llama.cpp` is CPU/edge-oriented and quantization-first by design, which makes for an interesting contrast).
- **Profiling: PyTorch Profiler and `nvidia-smi` / `nsys`** for GPU utilization and memory tracking — this is what generates the evidence behind your written analysis in Chapter 6.

4.4 3.4 Scope Boundary: What This Project Explicitly Does NOT Implement

Being explicit about this both keeps the project achievable in 6–9 weeks and demonstrates mature engineering judgment in interviews (knowing what to leave out is as important as knowing what to build):

- **No custom CUDA kernels.** You will use PyTorch’s existing attention/matmul kernels. Writing custom fused-attention kernels (like FlashAttention) is a legitimate follow-on project but a separate one.
- **No PagedAttention reimplementation.** You’ll implement a simpler (and less memory-efficient) cache allocation scheme, and your write-up should explicitly name this as the reason for part of the gap versus vLLM.
- **No multi-GPU/tensor-parallelism.** Single-GPU only. Distributed inference is a legitimate but distinct additional project.
- **No custom training or fine-tuning.** This project is inference-only; the model’s weights are used as-is (post-training quantization only).

5 Chapter 4: Phase-by-Phase Build Plan

5.1 4.1 Phase 0 — Environment and Baseline (2-3 days)

Goal: get a small open-weight model running with plain HuggingFace `generate()`, establish your baseline numbers before writing any custom code.

Steps: 1. Set up environment: PyTorch with CUDA support, `transformers`, `accelerate`. 2. Load a small open-weight model (1-3B range) with HuggingFace `transformers`. 3. Run baseline generation with the library's own `.generate()` method, measure tokens/sec and time-to-first-token using a simple wall-clock timer. 4. Install vLLM separately, run the same model through it, record its baseline numbers on the same hardware. This is your reference target for the entire project. 5. Record hardware specs (GPU model, VRAM, CUDA version) — every benchmark number in your final write-up must be tied to specific hardware, since these numbers are meaningless without that context.

Deliverable: a short markdown file with baseline numbers for both plain HuggingFace generation and vLLM, on the same model and hardware. This becomes the first row of your final comparison table.

5.2 4.2 Phase 1 — Naive Forward-Pass Generation, No Cache (3-4 days)

Goal: implement your own generation loop, without any caching, so you have a correct (if slow) baseline you fully understand and control.

```
import torch

def naive_generate(model, tokenizer, prompt, max_new_tokens):
    input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to(model.device)
    generated = input_ids

    for _ in range(max_new_tokens):
        with torch.no_grad():
            # Recomputes the FULL sequence every step -- this is the
            # inefficiency Phase 2 removes
            outputs = model(generated)
            next_token_logits = outputs.logits[:, -1, :]
            next_token = torch.argmax(next_token_logits, dim=-1, keepdim=True)
            generated = torch.cat([generated, next_token], dim=1)

        if next_token.item() == tokenizer.eos_token_id:
            break

    return tokenizer.decode(generated[0], skip_special_tokens=True)
```

What to measure here: confirm output text quality matches HuggingFace's own `.generate()` on the same prompt (correctness check before optimizing), and record how throughput degrades as generated length grows — you should see roughly quadratic scaling, which becomes the visual proof for why Phase 2 matters.

Deliverable: working naive generator, correctness-verified against the library baseline, with a throughput-vs-sequence-length plot showing the quadratic-ish growth curve.

5.3 4.3 Phase 2 — KV-Cache Implementation (1-1.5 weeks)

Goal: implement your own KV-cache, hooking into the model's attention layers, so each generation step only processes the newest token.

```
class KVCache:
    def __init__(self, num_layers, num_heads, head_dim, max_seq_len, batch_size, device,
dtype=torch.float16):
        cache_shape = (num_layers, batch_size, num_heads, max_seq_len, head_dim)
        self.k_cache = torch.zeros(cache_shape, device=device, dtype=dtype)
        self.v_cache = torch.zeros(cache_shape, device=device, dtype=dtype)
        self.current_len = 0
```

```

def update(self, layer_idx, new_k, new_v):
    # new_k, new_v shape: (batch_size, num_heads, new_tokens, head_dim)
    seq_len = new_k.shape[2]
    self.k_cache[layer_idx, :, :, self.current_len:self.current_len + seq_len, :] = new_k
    self.v_cache[layer_idx, :, :, self.current_len:self.current_len + seq_len, :] = new_v

def get(self, layer_idx):
    return (
        self.k_cache[layer_idx, :, :, :self.current_len, :],
        self.v_cache[layer_idx, :, :, :self.current_len, :]
    )

def advance(self, num_new_tokens):
    self.current_len += num_new_tokens

```

You'll need to modify the model's forward pass to accept and use this cache — the exact hook points depend on the specific model architecture you chose in Phase 0 (consult that model's `transformers` source code for its attention layer implementation; the pattern is consistent across Llama-family models but the exact class/method names vary by model family and library version).

```

def cached_generate(model, tokenizer, prompt, max_new_tokens, kv_cache):
    input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to(model.device)
    generated = input_ids
    current_input = input_ids # first pass processes the full prompt

    for step in range(max_new_tokens):
        with torch.no_grad():
            # current_input is ONLY the new token(s) after the first step --
            # this is the actual efficiency gain vs. Phase 1
            outputs = model(current_input, kv_cache=kv_cache)
            next_token_logits = outputs.logits[:, -1, :]
            next_token = torch.argmax(next_token_logits, dim=-1, keepdim=True)
            generated = torch.cat([generated, next_token], dim=-1)
            current_input = next_token # only feed the new token next time

        if next_token.item() == tokenizer.eos_token_id:
            break

    return tokenizer.decode(generated[0], skip_special_tokens=True)

```

What to measure here: throughput should now scale roughly linearly with sequence length instead of quadratically. Also measure and record memory consumption of the cache itself at various sequence lengths (this data feeds directly into your Chapter 2.3 discussion in the write-up).

Deliverable: working cached generator, with a side-by-side throughput comparison plot against Phase 1, and a memory-usage-vs-sequence-length plot for the cache itself.

5.4 4.4 Phase 3 — Continuous Batching Scheduler (1.5-2 weeks)

Goal: support multiple concurrent sequences, at different stages of generation, sharing GPU capacity efficiently, with sequences able to join and leave the running batch dynamically.

This is the most architecturally involved phase. Core components:

```

from dataclasses import dataclass, field
from enum import Enum

class SequenceState(Enum):
    WAITING = "waiting"
    RUNNING = "running"
    FINISHED = "finished"

@dataclass
class Sequence:
    seq_id: int
    prompt_tokens: list
    generated_tokens: list = field(default_factory=list)
    state: SequenceState = SequenceState.WAITING

```

```

max_new_tokens: int = 256
cache_slot: int = None # which slot in the batched KV-cache this sequence owns

class ContinuousBatchScheduler:
    def __init__(self, max_batch_size, kv_cache_pool):
        self.max_batch_size = max_batch_size
        self.waiting_queue = []
        self.running_batch = []
        self.kv_cache_pool = kv_cache_pool # pre-allocated cache slots

    def add_request(self, sequence: Sequence):
        self.waiting_queue.append(sequence)

    def step(self, model):
        # 1. Admit new sequences into the running batch if there's room
        while len(self.running_batch) < self.max_batch_size and self.waiting_queue:
            seq = self.waiting_queue.pop(0)
            seq.cache_slot = self.kv_cache_pool.allocate_slot()
            seq.state = SequenceState.RUNNING
            self.running_batch.append(seq)

        if not self.running_batch:
            return # nothing to do this step

        # 2. Run one generation step for every sequence currently in the batch
        # (this is a simplification -- a real implementation batches the
        # actual tensor forward pass across all running sequences at once,
        # rather than looping; the loop here is for clarity of the
        # scheduling logic, not the actual tensor operation)
        finished_this_step = []
        for seq in self.running_batch:
            next_token = self.generate_one_token(model, seq)
            seq.generated_tokens.append(next_token)
            if next_token == EOS_TOKEN_ID or len(seq.generated_tokens) >= seq.max_new_tokens:
                seq.state = SequenceState.FINISHED
                finished_this_step.append(seq)

        # 3. Remove finished sequences, freeing their cache slots for reuse
        for seq in finished_this_step:
            self.kv_cache_pool.free_slot(seq.cache_slot)
            self.running_batch.remove(seq)

        return finished_this_step

```

The comment in step 2 is important and worth restating: the naive version above loops over sequences one at a time, which does not actually get you the GPU utilization benefit of batching. The real engineering work in this phase is restructuring the forward pass to process all currently-running sequences as one batched tensor operation per step (stacking their current-step inputs, running one model call, then unstacking outputs) — a working but non-batched scheduler is a valid, honest **intermediate** milestone, but batched tensor execution is required for this phase to actually demonstrate the throughput benefit that's the entire point of Chapter 1.2's second technique. Budget real time for this restructuring — it's the single hardest engineering piece in the whole project.

What to measure here: GPU utilization (via `nvidia-smi` sampled during a run) under mixed workload (sequences of varying prompt/output lengths arriving at staggered times) — compare utilization with static batching (wait, run to completion, repeat) versus your continuous scheduler. This is the number that most directly demonstrates the value of this phase.

Deliverable: working scheduler supporting concurrent sequences with dynamic admission/eviction, a GPU utilization comparison (static vs. continuous batching) under a simulated mixed workload, and throughput (aggregate tokens/sec across all concurrent sequences) at several batch sizes.

5.5 4.5 Phase 4 — Weight Quantization (1-1.5 weeks)

Goal: implement weight-only post-training quantization (INT8, and optionally INT4) and measure the throughput/memory gains against the accuracy cost.

```
import torch
```

```

def quantize_weight_int8(weight: torch.Tensor):
    """Per-channel symmetric quantization of a weight matrix to int8."""
    scale = weight.abs().max(dim=1, keepdim=True).values / 127.0
    quantized = torch.round(weight / scale).clamp(-127, 127).to(torch.int8)
    return quantized, scale

def dequantize_weight_int8(quantized: torch.Tensor, scale: torch.Tensor):
    return quantized.to(torch.float16) * scale

class QuantizedLinear(torch.nn.Module):
    def __init__(self, original_linear: torch.nn.Linear):
        super().__init__()
        q_weight, scale = quantize_weight_int8(original_linear.weight.data)
        self.register_buffer("q_weight", q_weight)
        self.register_buffer("scale", scale)
        self.bias = original_linear.bias

    def forward(self, x):
        weight = dequantize_weight_int8(self.q_weight, self.scale)
        return torch.nn.functional.linear(x, weight, self.bias)

```

Note honestly: the `QuantizedLinear` module above dequantizes weights back to float16 before the matmul, which saves memory but not necessarily compute time (the matmul still runs in float16). Real inference speedups from quantization typically require int8/int4 matmul kernels that operate natively on the quantized representation (this is what libraries like `bitsandbytes` and `AutoGPTQ` actually implement). Getting a genuine speedup, not just a memory reduction, is a meaningfully harder step — a valid and honest scope choice for this project is to implement and clearly report the **memory** savings rigorously, attempt the **compute** speedup, and if a native low-precision matmul kernel proves out of reach in your timeline, state that explicitly in your write-up as a scope boundary (Chapter 3.4’s approach) rather than claiming a speedup you didn’t actually measure correctly.

What to measure here: memory footprint before/after quantization (weights and, separately, KV-cache if you also quantize the cache itself as a stretch goal), throughput before/after, and — critically — **accuracy/quality degradation**, measured with a real evaluation (e.g., perplexity on a held-out text sample, or accuracy on a small standard benchmark subset), not just eyeballing a few generated samples.

Deliverable: working INT8 quantization, a memory-savings measurement, a throughput measurement (with honest reporting per the note above), and a quantitative accuracy-degradation measurement.

5.6 4.6 Phase 5 — Rigorous Benchmarking Against vLLM (1-1.5 weeks)

Covered in full in Chapter 5, since the benchmarking methodology deserves its own dedicated treatment rather than being squeezed in here.

6 Chapter 5: Benchmarking Methodology

6.1 5.1 Why Methodology Matters More Than Results Here

A sloppy benchmark (different hardware, different prompts, warm cache on one system and cold on another, single run instead of averaged runs) produces numbers nobody should trust, including you. Since the entire value of this project is the honest comparison, the benchmark methodology is not a formality — it is the actual deliverable.

6.2 5.2 Fixed Variables (Must Be Identical Across All Comparisons)

- **Same GPU, same machine, same run session** — never compare numbers gathered on different hardware or across long time gaps without re-verifying baseline stability.
- **Same model weights** — your engine and vLLM must load the exact same checkpoint.
- **Same prompts and same generation length targets** — use a fixed, documented prompt set, not ad hoc prompts you happened to type.
- **Warm-up runs excluded** — the first 1-2 requests after loading a model are typically slower (CUDA kernel compilation/caching effects). Discard warm-up runs and only report steady-state numbers.

6.3 5.3 Metrics to Report (All of These, Not a Subset)

Metric	What It Captures	Why It Matters
Time-to-first-token (TTFT)	Latency before generation visibly starts	What a user actually perceives as “responsiveness”
Time-per-output-token (TPOT)	Steady-state generation speed	Determines how fast a long response streams in
Aggregate throughput (tokens/sec across all concurrent sequences)	System-wide efficiency under load	The number that matters for serving many users at once
Peak GPU memory usage	Memory efficiency	Determines how many concurrent sequences fit at all
GPU utilization (%) over the run	Whether the GPU is actually kept busy	Distinguishes “fast because efficient” from “fast because lightly loaded”
Accuracy/quality metric (perplexity or task accuracy)	Whether optimizations cost correctness	Required specifically for the quantization phase

6.4 5.4 Test Scenarios (Run All of These, Report All of Them)

1. **Single sequence, short prompt, short output** — baseline sanity check.
2. **Single sequence, long prompt, long output** — stresses the KV-cache specifically.
3. **Batch of N identical-length sequences arriving simultaneously** — tests batching efficiency under the easiest possible case.
4. **Batch of N sequences with staggered arrival times and varying lengths** — the realistic case, and the one where continuous batching’s advantage over static batching should be most visible.
5. **Increasing batch size until memory is exhausted** — establishes your engine’s practical capacity ceiling versus vLLM’s.

6.5 5.5 Statistical Rigor

- Run every scenario at least 5 times, report mean and standard deviation, not a single run’s number.
- Note and investigate any run with an outlier result rather than silently discarding it — an unexplained outlier is itself informative (thermal throttling, a background process, a bug).

6.6 5.6 The Actual Analysis Write-Up Structure

For each metric where your engine underperforms vLLM (expected, and fine), structure the explanation as:

1. **The measured gap** (e.g., “our engine achieves 34% of vLLM’s aggregate throughput at batch size 16”).
2. **The mechanistic reason**, backed by profiler evidence (e.g., “profiling shows our engine spends 40% of wall-clock time on Python-level list operations in the scheduler loop, versus vLLM’s fully compiled/batched C++ scheduling path”).
3. **What would close the gap**, even if you don’t implement it (e.g., “implementing PagedAttention-style block allocation would reduce our memory fragmentation, allowing a larger effective batch size”).

This three-part structure, repeated for every metric, is what turns a benchmark table into a genuinely research-grade analysis — and it’s the part of this project that will actually come up, nearly verbatim, in a technical interview.

7 Chapter 6: Expected Results and Honest Framing

7.1 6.1 What You Should Expect to Find (Calibrating Expectations Before You Start)

Based on the architecture of what you're building versus what vLLM actually implements:

- Your engine will very likely be **significantly slower** than vLLM at higher batch sizes — this is expected and fine, because vLLM's PagedAttention specifically targets memory efficiency at scale, which you are explicitly not reimplementing (Chapter 3.4).
- At batch size 1, the gap should be much smaller, since many of vLLM's optimizations exist specifically to handle concurrent, variable-length request scheduling — a domain where a simpler implementation is less disadvantaged.
- Your continuous batching scheduler, even simplified, should still show a clear GPU utilization improvement over naive static batching — this comparison (your continuous vs. your own static/naive baseline) is arguably more pedagogically valuable than the vLLM comparison, because it isolates the specific technique's contribution without comparing across two completely different codebases.

7.2 6.2 If Your Results Don't Match These Expectations

If your engine is somehow competitive with vLLM at high batch sizes, that's a signal to double check your benchmark methodology (Chapter 5.2) before celebrating — it's far more likely you have an unfair comparison (e.g., vLLM running with a suboptimal configuration, or your engine's numbers reflecting a warm cache that vLLM's don't) than that you've independently matched a mature production system's engineering.

7.3 6.3 Why an Honest "This Didn't Work" Result Is Still a Strong Deliverable

If you attempt the compute-side quantization speedup (Chapter 4.5) and it doesn't materialize into a real speedup because implementing a native low-precision matmul kernel was out of reach in your timeline, reporting that clearly — with the profiling data showing *why* it didn't speed things up — is a legitimate and interview-worthy outcome. Research and infra engineering is full of attempted optimizations that don't pan out for well-understood reasons; being able to explain precisely why something didn't work is a stronger signal than a suspiciously clean success across every single metric.

8 Chapter 7: Timeline Summary

Phase	Duration	Deliverable
Phase 0: Environment & Baseline	2-3 days	Baseline numbers file (HuggingFace + vLLM on same hardware)
Phase 1: Naive Generation	3-4 days	Working uncached generator + correctness verification
Phase 2: KV-Cache	1-1.5 weeks	Cached generator + throughput/memory plots
Phase 3: Continuous Batching	1.5-2 weeks	Scheduler + GPU utilization comparison
Phase 4: Quantization	1-1.5 weeks	INT8 implementation + accuracy/throughput measurements
Phase 5: Benchmarking & Write-up	1-1.5 weeks	Full comparison report against vLLM
Total	6-9 weeks	Complete engine + repo + written analysis

Phase 3 is the long pole — budget slack there specifically, since the batched-tensor-execution restructuring (Chapter 4.4) is the hardest single piece of engineering in the project and is the most likely place to run over estimate.

9 Chapter 8: Repository Structure

```
microinfer/
├── README.md           # Project overview, results summary, how to reproduce
├── requirements.txt
├── src/
│   ├── model_loader.py # Loads base model + tokenizer
│   ├── naive_generate.py # Phase 1
│   ├── kv_cache.py      # Phase 2
│   ├── cached_generate.py # Phase 2
│   ├── scheduler.py     # Phase 3
│   ├── quantization.py  # Phase 4
│   └── engine.py        # Ties everything together into one callable engine
├── benchmarks/
│   ├── run_benchmark.py # Reproducible benchmark runner
│   ├── scenarios.py     # The 5 test scenarios from Chapter 5.4
│   └── results/         # Raw benchmark output (CSV/JSON), committed for reproducibility
├── analysis/
│   ├── ANALYSIS.md     # The written gap-analysis from Chapter 5.6
│   └── plots/          # Generated comparison charts
└── tests/
    └── test_correctness.py # Verifies output matches HuggingFace baseline
```

README.md must include, at minimum: a one-paragraph summary of what the project does, a results table (your engine vs. vLLM, key metrics), a link to `ANALYSIS.md` for the full write-up, and exact reproduction instructions (hardware used, exact commands to run). This README is very likely the first and possibly only part of the repo a recruiter or interviewer actually reads before the interview itself — it needs to make your strongest points in under two minutes of reading.

10 Chapter 9: Interview Preparation — Turning This Project Into Talking Points

10.1 9.1 Questions You Should Be Able to Answer Cold

Prepare precise, specific answers (with numbers, not vague impressions) for:

- “Walk me through what happens, mechanically, when you generate one token with your cache versus without it.”
- “Why is your engine slower than vLLM at batch size 32 but competitive at batch size 1?”
- “What would you need to implement to close that gap, and why haven’t you?”
- “What surprised you during this project?” (Have a real, specific answer — interviewers can tell a rehearsed non-answer from genuine engineering experience.)
- “What was the hardest bug you hit, and how did you find it?” (This should reference actual profiler output or actual debugging steps you took, not a hypothetical.)

10.2 9.2 The Failure-Log Habit (Start This From Day One)

Keep a running plain-text or markdown log, updated as you go, of: what you tried, what broke, what you saw in the profiler/logs, and what you changed. This becomes both your `ANALYSIS.md` source material and your interview prep material simultaneously — do not try to reconstruct this from memory after the fact, since the specific, precise details are what make the difference in how it lands.

11 Appendix A: Glossary

KV-cache — stored Key/Value projections from previous tokens, reused across generation steps to avoid recomputation (Chapter 2.1-2.2).

Continuous batching — a scheduling technique allowing sequences to join and leave an active batch dynamically, rather than waiting for a fixed batch to fully complete (Chapter 1.2, Chapter 4.4).

PagedAttention — vLLM’s technique for allocating KV-cache memory in fixed-size, non-contiguous blocks (analogous to OS virtual memory paging), avoiding the fragmentation and waste of pre-allocated, padded cache tensors. Explicitly out of scope for this project’s own implementation (Chapter 3.4), but essential to understand as the primary reason for vLLM’s advantage at scale.

Time-to-first-token (TTFT) — latency between sending a request and receiving the first generated token.

Time-per-output-token (TPOT) — steady-state per-token generation latency after the first token.

Weight-only quantization — reducing the numeric precision of stored model weights while keeping activations at higher precision during computation (Chapter 2.5, Chapter 4.5).

12 Appendix B: Honest Caveats on This Document

Model names, specific library versions, and exact hardware recommendations are deliberately left general rather than pinned to specific current releases, because open-weight model availability, licensing terms, and library APIs (`transformers` , `vLLM`) change on a timescale of months. Before starting Phase 0, do a quick check of currently available small open-weight models and current `vLLM` installation requirements, since the specific choice matters less than the architecture and methodology described throughout this document, which remain valid regardless of which specific model you choose.